

수업 06 · 2026-10-21

HTML/CSS/React/MCP

HTML과 CSS로 화면을 읽고 고친 뒤 React의 화면 부품을 살펴봅니다. 별도 MCP 예제로 도구의 요청과 응답을 확인합니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

처음 시작하기 · HTML/CSS/React/MCP

1. ZIP을 일반 폴더에 풀고 examples/course의 data·work·code 위치를 유지합니다. 배포 ZIP은 course 폴더부터 열 수 있습니다.
2. [교안](#)에서 개념을 읽고 [실습지](#)의 A01부터 진행합니다.
3. work 파일만 수정하고 원본은 보존합니다. 같은 파일을 저장·다시 열어 확인합니다.
4. 개인 실행을 못 했으면 예상과 강사 실행 결과를 대조하고 확인 방법을 정확히 적습니다.

실행 준비

브라우저와 텍스트 편집기가 기본입니다. Python 명령의 현재 폴더는 압축을 푼 course입니다. Windows에서 python 대신 py만 있으면 강사의 확인 아래 py -3로 대체합니다.

```
python code/mcp_client.py
```

실행하지 못하면 파일 경로·명령·오류 문구를 기록해 도움을 요청합니다. 코드 상자를 보는 것은 Python 실행이 아닙니다.

React 19.1.1과React DOM 19.1.1을 로컬 번들로 제공합니다. 설치와 전체 앱 작성은 필수가 아닙니다. MCP는 2025-11-25 명시 버전의 소규모 STUDIO 예제이며 최신 전체 규격·HTTP·인증·AI 호스트 연결을 구현했다는 뜻이 아닙니다. React와 MCP 자동 연결은 없습니다.

수업 전, 이 세 가지부터 확인하세요

앞선 내용이 기억나지 않으면 아래 질문부터 읽어 보세요. 먼저 답을 적고 풀이와 비교하면 됩니다.

역할

질문: HTML, CSS, JavaScript는 같은 일을 하나요?

풀이: HTML은 내용과 구조, CSS는 모양, JavaScript는 동작을 담당합니다. 버튼 글자와 버튼 색, 클릭 뒤 값 변화를 나눠 봅니다.

상태

질문: 센서 카드 값을 바꾼 뒤 새로고침하면 꼭 남나요?

풀이: 화면의 현재 상태와 저장된 데이터를 구별합니다. 저장 기능이 없다면 초기 입력으로 돌아갈 수 있습니다.

연결

질문: React 화면이 있으면 MCP 도구도 자동으로 붙나요?

풀이: 화면과 도구 요청 경로는 별도로 연결하고 확인해야 합니다. 이 수업의 제공 예제는 두 역할을 구별해 읽습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A15	A14
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 사례와 풀이를 읽고, 이번 교시에 정한 활동을 직접 해 봅니다. 활동 결과를 확인한 뒤에는 추가 문제를 풀어 봅니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

이전 Python·Git 기초를 짧게 복습하며 준비된 작은 예제를 읽고 수정합니다. **최신 운영은 50분 × 6교시, 순수 수업 300분**입니다. 설명 보충은 필수 개념이며 독립 과제는 추가 복습입니다. 추가 문제는 수업 후 복습에 사용합니다.

React 19.1.1과React DOM 19.1.1을 로컬 번들로 제공합니다. 설치와 전체 앱 작성은 필수가 아닙니다. MCP는 2025-11-25 명시 버전의 소규모 STUDIO 예제이며 최신 전체 규격·HTTP·인증·AI 호스트 연결을 구현했다는 뜻이 아닙니다. React와 MCP 자동 연결은 없습니다.

전체 코드와 모든 줄의 뜻·실행 방법은 [코드 해설](#)에 있습니다. 작성 소스와 생성된 번들·잠금 파일을 구분합니다.

1교시 · 웹 문서의 내용

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 풉니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A01 · 파일과 브라우저의 관계

사례로 이해하기 · 문서의 의미와 화면 표시를 분리하기

실무 상황

작업장 안내판의 제목을 바꿨는데 브라우저 탭 이름은 예전대로입니다. 파일 이름도 달라지지 않았습니다. 제목이라는 말이 세 곳에 쓰이지만 파일 이름, 탭 이름, 본문 제목은 각각 다른 정보입니다. 어디를 바꿨고 어떤 결과를 기대했는지 먼저 정해야 합니다.

쉬운 풀이

HTML은 문서 내용과 의미를 표시하는 글입니다. 태그는 내용을 감싸며 역할을 붙이는 표시입니다. h1은 본문의 큰 제목 역할이고 title은 브라우저 탭의 이름입니다. 속성은 태그에 붙이는 추가 정보이며 id는 프로그램이 특정 요소를 찾는 이름입니다. 편집기는 글을 고치는 곳, 브라우저는 문서를 읽어 화면으로 보여 주는 곳입니다. 확장자 .html은 파일 종류를 나타내며 .html.txt와 구별합니다.

완성 예와 단계 풀이

아래 문서를 안내.html로 저장합니다. 꺾쇠와 닫는 태그를 유지하고 한글 내용만 읽습니다.

```
<!doctype html>

<html lang="ko"><head><meta charset="utf-8"><title>작업장</title></head>

<body><h1>센서 안내</h1><button id="check">값 확인</button></body></html>
```

첫 줄은 HTML 문서 형식, lang은 한국어 정보, charset은 한글을 읽는 문자 규칙입니다. head는 문서 정보, body는 보이는 내용입니다. 브라우저에서 탭은 작업장, 본문은 센서 안내, 버튼은 값 확인으로 보입니다. 버튼은 아직 동작 코드가 없으므로 누른다고 센서 값을 읽지 않습니다. 저장한 같은 파일을 열고 수정 뒤 새로고침합니다.

독립 추가 복습 과제

탭은 "공장 A", 본문 제목은 "온도 확인", 버튼은 "기록 조회"로 작성하되 버튼 id는 check를 유지하세요. 이어서 id까지 바꾸면 뒤의 JavaScript 연결에 어떤 확인이 필요한지 설명하세요.

답과 대표 오답

title과 h1과 버튼 사이 글자를 각각 바꾸며 id는 유지합니다. "글자 크기가 같으면 제목 역할도 같다"는 답은 의미와 모양을 혼동합니다. "버튼이 보이면 데이터 연결도 됐다"는 답에는 동작 확인이 빠졌습니다.

확인 질문과 해설

① 저장만 하면 화면이 자동 갱신되나요? 제공 정적 파일은 새로고침이 필요합니다. ② id는 사용자 문구인가요? 내부에서 찾는 이름입니다. ③ 한글이 깨지면 무엇을 확인하나요? 파일 저장 인코딩과 charset, 실제 열린 경로를 확인합니다.

뜻과 비유. 브라우저는 HTML 문서를 읽어 화면으로 보여 주는 프로그램입니다. 파일을 편집하는 창과 결과를 보는 창은 역할이 다릅니다.

왜 필요한가. 어디에 썼는지와 무엇을 열었는지를 구분해야 저장한 변경을 다시 확인할 수 있습니다.

예를 따라 읽기. work/static/index.html을 편집하고 같은 파일을 브라우저로 열면 제공 안내판을 볼 수 있습니다.

헛갈리는 반례. 코드 편집기에 글자를 바꾼 것만으로 이미 열린 화면이 자동 갱신되는 것은 아닙니다.

직접 적용. course 폴더를 압축 풀기 → work/static/index.html 경로 적기 → 브라우저에서 안내판 열기 → 편집·저장·새로고침 순서를 말하기

설명할 질문. 편집 창과 브라우저 화면은 어떻게 다른가요?

막혔을 때. 확장자가.txt로 붙었으면 파일 이름과 종류를 확인합니다. ZIP 안의 미리 보기에서 편집하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A01](#)와 [활동 원문](#)에 있습니다.

A01 · 직접 해보기

A01 · 파일과 브라우저의 관계

목적

어디에 썼는지와 무엇을 열었는지를 구분해야 저장한 변경을 다시 확인할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [index.html](#), [A01_input.md](#)

작업: [A01_notes.md](#), [index.html](#)

실제 입력

work/static/index.html을 편집하고 같은 파일을 브라우저로 열면 제공 안내판을 볼 수 있습니다.

단계

1. course 폴더를 압축 풀기
2. work/static/index.html 경로 적기
3. 브라우저에서 안내판 열기
4. 편집·저장·새로고침 순서를 말하기

예상 결과

안내판 화면과 정확한 파일 경로

확인 방법

편집 창과 브라우저 화면은 어떻게 다른가요?

오류 해결

확장자가.txt로 붙었으면 파일 이름과 종류를 확인합니다. ZIP 안의 미리 보기에서 편집하지 않습니다.

힌트

브라우저는 HTML 문서를 읽어 화면으로 보여 주는 프로그램입니다. 파일을 편집하는 창과 결과를 보는 창은 역할이 다릅니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 편집 창과 브라우저 화면은 어떻게 다른가요? → ____
- 입력 파일과 내가 바꾼 위치: ____
- 실행 전 예상 결과: ____
- 실제 관찰 결과와 확인 방법: ____
- 예상과 달랐던 점·복구·남은 질문: ____

수정 전 예상: _____

확인한 결과와 이유: _____

A02 · HTML 태그와 문서 의미

뜻과 비유. HTML은 제목·문단·버튼 같은 내용의 의미를 표시하는 문서 형식입니다. 여는 태그와 닫는 태그 사이의 내용이 하나의 요소가 됩니다.

왜 필요한가. 보기에 큰 글자를 만드는 것과 문서의 제목이라는 의미를 주는 것을 구별해야 내용 구조를 읽을 수 있습니다.

예를 따라 읽기. `<h1>입구 센서 안내판</h1>`에서 h1은 주요 제목, 안의 글자는 화면 내용입니다.

헷갈리는 반례. 모든 글자를 h1으로 만들면 크기가 같아질 수 있어도 제목과 설명의 관계가 흐려집니다.

직접 적용. 전체 HTML에서 h1·p·button 찾기 → 여는 태그와 닫는 태그 짝 표시하기 → 제목 글자만 창고 센서 안내판으로 바꾸기 → 저장 후 어떤 내용이 바뀌었는지 확인하기

설명할 질문. h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요?

막혔을 때. 태그 꺾쇠를 지웠다면 원본의 해당 줄과 비교합니다. 제목 문구와 title 탭 이름은 다른 위치입니다.

입력·답칸·확인 순서는 [실습지의 A02](#)와 [활동 원문](#)에 있습니다.

A02 · 직접 해보기

A02 · HTML 태그와 문서 의미

목적

보기에 큰 글자를 만드는 것과 문서의 제목이라는 의미를 주는 것을 구별해야 내용 구조를 읽을 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [index.html](#), [A02_input.md](#)

작업: [A02_notes.md](#), [index.html](#)

실제 입력

`<h1>입구 센서 안내판</h1>`에서 h1은 주요 제목, 안의 글자는 화면 내용입니다.

단계

1. 전체 HTML에서 h1·p·button 찾기
2. 여는 태그와 닫는 태그 짝 표시하기
3. 제목 글자만 창고 센서 안내판으로 바꾸기
4. 저장 후 어떤 내용이 바뀌었는지 확인하기

예상 결과

제목이 바뀐 작업 HTML

확인 방법

h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요?

오류 해결

태그 꺾쇠를 지웠다면 원본의 해당 줄과 비교합니다. 제목 문구와 title 탭 이름은 다른 위치입니다.

힌트

HTML은 제목·문단·버튼 같은 내용의 의미를 표시하는 문서 형식입니다. 여는 태그와 닫는 태그 사이의 내용이 하나의 요소가 됩니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: h1과 CSS 글자 크기는 왜 같은 역할이 아닌가요? → _____

- 입력 파일과 내가 바꾼 위치: _____

- 실행 전 예상 결과: _____

- 실제 관찰 결과와 확인 방법: _____

- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 속성·연결·접근성

이 교시의 다른 예와 코드 해설 열기

2교시 · CSS의 모양과 읽기

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A04 · CSS 선택자와 선언

사례로 이해하기 · 선택자와 상자 모델로 읽기 편한 화면 만들기

실무 상황

센서 카드의 글자가 테두리에 붙어 읽기 어렵습니다. 카드 사이 간격도 좁습니다. 글자만 띄우려다가 페이지 전체를 오른쪽으로 밀면 작은 화면에서 내용이 잘립니다. 어떤 상자의 어느 공간을 바꾸는지 먼저 그려 봅니다.

쉬운 풀이

CSS는 문서의 모양을 정하는 규칙입니다. 선택자는 꾸밀 대상을 고르는 이름이고 선언은 속성과 값의 짝입니다. 중괄호 안의 `color: teal;`은 글자색을 청록으로 정합니다. 세미콜론은 선언의 끝입니다. 상자에서 `content`는 내용, `padding`은 내용과 테두리 사이, `border`는 테두리, `margin`은 다른 상자와의 바깥 간격입니다. 색과 여백은 온도 데이터의 의미를 바꾸지 않습니다.

완성 예와 풀이

앞 문서의 `head` 안에 `style` 태그로 아래 규칙을 넣습니다.

```
body { margin: 16px; font-family: sans-serif; }  
h1 { color: teal; }  
button { padding: 12px; margin-top: 8px; }  
button:focus-visible { outline: 3px solid black; }
```

`body` 규칙은 전체 가장자리와 글꼴, `h1` 규칙은 제목만, `button` 규칙은 버튼 안팎 공간을 조절합니다. 마지막 규칙은 키보드로 이동할 때 초점을 표시합니다. 저장 후 새로고침하고 `Tab` 키로 버튼에 이동합니다. 초점은 지금 키보드로 조작할 대상이며 색 표시만으로 정상 상태를 전달하지 않듯 초점도 보이게 제공합니다. 별도 `style.css`를 쓰면 HTML의 `link` 경로가 같은 파일을 가리키는지도 확인합니다.

독립 추가 복습 과제

버튼 글자 주변 공간만 늘리고 버튼과 제목 사이 공간은 유지하려면 어떤 속성을 바꾸나요? 다음으로 정상 상태를 녹색 원 하나로만 보여 주는 화면을 글자와 함께 표현하는 문장을 작성하세요.

답과 대표 오답

안쪽 공간은 `padding`입니다. 상태 문장은 “정상: 제공된 가상 온도가 기준 미만입니다”처럼 뜻을 적습니다. `margin`을 늘리는 답은 바깥 공간을 바꿉니다. “색을 더 진하게 하면 의미가 전달된다”는 답은 색을 구별하기 어려운 상황을 해결하지 못합니다.

확인 질문과 해설

① CSS 변경으로 센서값이 변하나요? 표시 모양만 달라집니다. ② 적용이 안 되면 무엇부터 보나요? 저장·경로·선택자·새로 고침을 확인합니다. ③ 좁은 화면도 확인하는 이유는 무엇인가요? 읽기 폭과 넘침은 큰 화면에서 보이지 않을 수 있습니다.

뜻과 비유. CSS는 어떤 요소에 어떤 모양을 적용할지 쓰는 규칙입니다. 선택자는 대상을 고르고, 속성과 값은 색·크기·간격을 정합니다.

왜 필요한가. 내용과 모양을 나누면 같은 제목을 유지하면서 여러 화면 크기에 맞게 조정할 수 있습니다.

예를 따라 읽기. `h1 { color: #126d65; }`는 제목 글자의 색을 정합니다. #126d65는 색을 나타내는 표기입니다.

헷갈리는 반례. 색을 바꿔도 센서 온도의 데이터 값이 바뀌는 것은 아닙니다.

직접 적용. style.css에서 h1 규칙 찾기 → color와 값의 경계 표시하기 → 색을 #75421b로 바꾸기 → 같은 제목 내용이 유지되는지 비교하기

설명할 질문. CSS 값을 바꿨을 때 무엇이 유지돼야 하나요?

막혔을 때. CSS가 적용되지 않으면 HTML의 href와 파일 이름이 일치하는지 먼저 확인합니다.

입력·답칸·확인 순서는 [실습지의 A04](#)와 [활동 원문](#)에 있습니다.

A04 · 직접 해보기

A04 · CSS 선택자와 선언

목적

내용과 모양을 나누면 같은 제목을 유지하면서 여러 화면 크기에 맞게 조정할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [style.css](#), [style.css](#), [index.html](#), [A04_input.md](#)

작업: [A04_notes.md](#), [style.css](#)

실제 입력

`h1 { color: #126d65; }`는 제목 글자의 색을 정합니다. #126d65는 색을 나타내는 표기입니다.

단계

1. style.css에서 h1 규칙 찾기
2. color와 값의 경계 표시하기
3. 색을 #75421b로 바꾸기
4. 같은 제목 내용이 유지되는지 비교하기

예상 결과

색만 달라진 제목과 수정한 규칙

확인 방법

CSS 값을 바꿨을 때 무엇이 유지돼야 하나요?

오류 해결

CSS가 적용되지 않으면 HTML의 href와 파일 이름이 일치하는지 먼저 확인합니다.

힌트

CSS는 어떤 요소에 어떤 모양을 적용할지 쓰는 규칙입니다. 선택자는 대상을 고르고, 속성과 값은 색·크기·간격을 정합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: CSS 값을 바꿨을 때 무엇이 유지돼야 하나요? → _____

- 입력 파일과 내가 바꾼 위치: _____

- 실행 전 예상 결과: _____

- 실제 관찰 결과와 확인 방법: _____

- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

A05 · 상자·여백·읽기 폭

뜻과 비유. 브라우저의 요소는 화면 공간을 차지하는 상자로 배치됩니다. padding은 안쪽 간격, margin은 바깥 간격이며 max-width는 너무 넓어지지 않게 제한합니다.

왜 필요한가. 문장이 화면 끝까지 길어지거나 요소가 붙어 있으면 내용을 읽기 어렵습니다. 간격도 정보 전달에 영향을 줍니다.

예를 따라 읽기. main의 padding:24px을 32px로 바꾸면 안내판 내용과 바깥 영역 사이의 안쪽 간격이 커집니다.

햇갈리는 반례. 글자를 작게 줄여 모든 것을 한 화면에 넣는 방식은 초심자의 읽기를 어렵게 할 수 있습니다.

직접 적용. main의 세 배치 규칙 찾기 → padding과 margin 을 그림으로 구분하기 → padding 한 값을 바꿔 비교하기 → 글자 크기와 읽기 폭의 영향을 설명하기

설명할 질문. 안쪽 여백과 바깥 여백은 어떻게 구분하나요?

막혔을 때. 차이가 안 보이면 같은 파일을 저장했는지와 브라우저 새로고침 여부를 확인합니다.

입력·답칸·확인 순서는 [실습지의 A05](#)와 [활동 원문](#)에 있습니다.

A05 · 직접 해보기

A05 · 상자·여백·읽기 폭

목적

문장이 화면 끝까지 길어지거나 요소가 붙어 있으면 내용을 읽기 어렵습니다. 간격도 정보 전달에 영향을 줍니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [style.css](#), [A05_input.md](#)

작업: [A05_notes.md](#), [style.css](#)

실제 입력

main의 padding:24px을 32px로 바꾸면 안내판 내용과 바깥 영역 사이의 안쪽 간격이 커집니다.

단계

1. main의 세 배치 규칙 찾기
2. padding과 margin 을 그림으로 구분하기
3. padding 한 값을 바꿔 비교하기
4. 글자 크기와 읽기 폭의 영향을 설명하기

예상 결과

간격 수정 전후 관찰 표

확인 방법

안쪽 여백과 바깥 여백은 어떻게 구분하나요?

오류 해결

차이가 안 보이면 같은 파일을 저장했는지와 브라우저 새로고침 여부를 확인합니다.

힌트

브라우저의 요소는 화면 공간을 차지하는 상자로 배치됩니다. padding은 안쪽 간격, margin 은 바깥 간격이며 max-width 는 너무 넓어지지 않게 제한합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 안쪽 여백과 바깥 여백은 어떻게 구분하나요? → ____

- 입력 파일과 내가 바꾼 위치: ____

- 실행 전 예상 결과: ____

- 실제 관찰 결과와 확인 방법: ____

- 예상과 달랐던 점·복구·남은 질문: ____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 작은 화면과 상태의 표시

이 교시의 다른 예와 코드 해설 열기

3교시 · 버튼과 데이터 변화

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 풉니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A08 · 데이터와 화면 내용 연결

뜻과 비유. 데이터는 프로그램이 보여 줄 값이며 화면 요소는 그 값을 담는 자리입니다. `textContent`는 글자를 넣는 방법으로 오늘의 단순 숫자·문구 표시에 사용합니다.

왜 필요한가. 어느 파일의 값이 어느 화면 자리로 이동하는지 알면 작은 변경을 스스로 확인할 수 있습니다.

예를 따라 읽기. `data.js`의 `temperature_c`를 24.5로 바꾼 뒤 클릭하면 온도 자리에 24.5가 표시됩니다.

헛갈리는 반례. HTML에 처음 적힌 22.5와 클릭 후 `data.js`에서 읽은 값은 출처가 다르므로 수정 위치를 구분합니다.

직접 적용. data.js의 세 필드 읽기 → 온도 하나를 24.5로 바꾸기 → 새로고침 직후와 클릭 후 비교하기 → 초기 HTML 값과 동적 값의 출처 쓰기

설명할 질문. 새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요?

막혔을 때. 값이 그대로면 클릭까지 수행했는지 봅니다. 초기 HTML 값만 확인하고 변경 실패라고 판단하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A08](#)과 [활동 원문](#)에 있습니다.

A08 · 직접 해보기

A08 · 데이터와 화면 내용 연결

목적

어느 파일의 값이 어느 화면 자리로 이동하는지 알면 작은 변경을 스스로 확인할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [data.js](#), [app.js](#), [index.html](#), [A08_input.md](#)

작업: [A08_notes.md](#), [data.js](#), [app.js](#), [index.html](#)

실제 입력

data.js의 temperature_c를 24.5로 바꾼 뒤 클릭하면 온도 자리에 24.5가 표시됩니다.

단계

1. data.js의 세 필드 읽기
2. 온도 하나를 24.5로 바꾸기
3. 새로고침 직후와 클릭 후 비교하기
4. 초기 HTML 값과 동적 값의 출처 쓰기

예상 결과

입력 24.5와 클릭 후 표시 값의 대응

확인 방법

새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요?

오류 해결

값이 그대로면 클릭까지 수행했는지 봅니다. 초기 HTML 값만 확인하고 변경 실패라고 판단하지 않습니다.

힌트

데이터는 프로그램이 보여 줄 값이며 화면 요소는 그 값을 담는 자리입니다. `textContent`는 글자를 넣는 방법으로 오늘의 단순 숫자·문구 표시에 사용합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 새로고침 직후와 클릭 뒤 값이 다른 이유는 무엇인가요? → ____
- 입력 파일과 내가 바꾼 위치: ____
- 실행 전 예상 결과: ____
- 실제 관찰 결과와 확인 방법: ____
- 예상과 달랐던 점·복구·남은 질문: ____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: JavaScript와 이벤트 / 오타를 찾고 필요한 부분만 고치기

이 교시의 다른 예와 코드 해설 열기

4교시 · React 완성 부품

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 풉니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A11 · props와 제공 데이터 수정

뜻과 비유. props는 부모가 부품에 전달하는 입력입니다. 오늘은 장소를 전달하고 온도의 첫 값을 제공 데이터에서 읽는 작은 완성 예제를 사용합니다.

왜 필요한가. 부품을 처음부터 새로 만들지 않아도 어떤 입력을 바꾸면 어느 화면이 달라지는지 확인할 수 있습니다.

예를 따라 읽기. `work/react/data.js`에서 `location`을 찾고, `temperature_c`를 24.5로 바꾸고 새로고침하면 부품의 내용이 바뀝니다.

햇갈리는 반례. JSX의 중괄호는 여기서 JavaScript 값을 넣는 자리이며 CSS 규칙의 중괄호와 역할이 다릅니다.

직접 적용. 작업 React 폴더 열기 → `data.js`의 장소와 온도 수정하기 → 새로고침 해두 표시 비교하기 → props로 전달되는 장소 경로 쓰기

설명할 질문. 같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요?

막혔을 때, 수정한 data.js가 code인지 work인지 확인합니다. 변경 값을 읽는 곳은 현재 열린 HTML 과 같은 폴더입니다.

입력·답칸·확인 순서는 [실습지의 A11](#)와 [활동 원문](#)에 있습니다.

작업 화면으로 전환하기 · A10 → A11

탐색기에서 [course/work/react/index.html](#)을 브라우저로 엽니다. 주소에 **work/react/index.html**이 있는지 확인하고 기록합니다. 같은 폴더의 [data.js](#)를 수정·저장한 뒤 이 작업 탭을 새로고침합니다. 장소 [창고](#), 온도 [24.5](#)가 보이는지 비교합니다. A10의 code/react 탭은 원본이므로 작업 수정이 반영되지 않습니다. App.jsx 재빌드는 선택입니다.

A11 · 직접 해보기

A11 · props와 제공 데이터 수정

목적

부품을 처음부터 새로 만들지 않아도 어떤 입력을 바꾸면 어느 화면이 달라지는지 확인할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [data.js](#), [App.jsx](#), [A11_input.md](#)

작업: [A11_notes.md](#), [data.js](#), [작업 index.html](#)

실제 입력

work/react/data.js에서 location을 창고,temperature_c를 24.5로 바꾸고 새로고침하면 부품의 내용이 바뀝니다.

단계

1. 작업 React 폴더 열기
2. data.js의 장소와 온도 수정하기
3. 새로고침 해두 표시 비교하기
4. props로 전달되는 장소 경로 쓰기

예상 결과

장소·온도 수정본과 결과

확인 방법

같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요?

오류 해결

수정한 data.js가 code인지 work인지 확인합니다. 변경 값을 읽는 곳은 현재 열린 HTML 과 같은 폴더입니다.

힌트

props는 부모가 부품에 전달하는 입력입니다. 오늘은 장소를 전달하고 온도의 첫 값을 제공 데이터에서 읽는 작은 완성 예제를 사용합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 같은 부품을 다른 장소에 쓰려면 어떤 입력을 바꿀까요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

작업 화면으로 전환하기 · A10 → A11

1. A10에서 본 `code/react/index.html` 화면을 닫거나 원본 탭이라고 표시합니다.
2. 탐색기에서 `course/work/react/index.html`을 브라우저로 엽니다. 주소 표시줄에 **work/react/index.html**이 포함됐는지 확인하고 실제 주소를 답칸에 적습니다.
3. 같은 `work/react` 폴더의 `data.js`를 편집기에서 열어 장소와 온도만 수정하고 저장합니다.
4. 2단계의 작업 화면을 새로고침합니다. 장소 **창고**, 온도 **24.5**가 보이는지 비교합니다.

원본 탭을 새로고침하면 작업값이 반영되지 않습니다. 두 파일 모두 이름은 `index.html`이므로 주소의 **code / work**까지 확인합니다. 소스 `App.jsx` 재빌드는 선택 확장입니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: React와 컴포넌트 / 상태와 화면의 다시 그리기

이 교사의 다른 예와 코드 해설 열기

5교시 · MCP 요청과 응답

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A13 · MCP의 호스트·클라이언트·서버

사례로 이해하기 · 직접 연결과 MCP 연결을 비교한 뒤 권한 정하기

실무 상황

온도 안내판이 기록을 보여 주는 방법에는 파일을 직접 읽는 경로, 서버 API를 호출하는 경로, AI 도구가 MCP 서버를 통해 기록을 조회하는 경로가 있습니다. 화면 모양은 비슷해도 중간 프로그램과 권한은 다릅니다. 연결을 그렸다는 사실만으로 실제 호출이 구현되었다고 보고하지 않습니다.

쉬운 풀이

MCP는 AI 애플리케이션이 도구와 자료에 접근할 때 주고받는 메시지의 공통 약속입니다. 호스트는 AI 애플리케이션, 클라이언트는 서버와 통신하는 부분, 서버는 도구와 자료를 제공하는 프로그램입니다. 직접 파일 읽기는 공통 프로토콜 없이 해당 파일 구조를 알아야 합니다. MCP는 초기화·도구 목록·호출 같은 약속을 공유하지만 모든 서버가 같은 자료나 권한을 갖지는 않습니다. 연결 가능성과 쓰기 허용도 별개입니다.

완성 예와 단계 풀이

오늘은 mcp_client.py가 로컬 mcp_server.py를 실행하여 표준 입력과 표준 출력으로 메시지를 주고받습니다. initialize 요청에 id 1이 있고 initialized 알림에는 id가 없습니다. tools/list가 get_reading의 입력 이름 sensor_id를 알려 줍니다. tools/call로 S01을 요청하면 22.5가 담긴 가상 기록이 돌아옵니다. S02는 존재하지만 미측정 null이며 S99는 Unknown sensor와 isError입니다. 이 서버는 읽기 도구만 제공하며 네트워크 요청, React 갱신, 실제 AI 추론을 수행하지 않습니다. 실제 호스트의 권한 통제 전체를 재현한 프로그램도 아닙니다.

독립 추가 복습 과제

“HTML의 고정 숫자”, “data.js의 가상 값”, “MCP get_reading의 응답”을 자료 출처와 필요한 연결 단계로 비교하세요. 이어서 센서 조회에 계정 삭제 도구 권한까지 요구하는 서버가 있다면 허용 범위를 어떻게 판단할지 쓰세요.

답과 대표 오답

고정 숫자는 HTML, 가상 값은 데이터 파일, MCP 응답은 서버 기록과 호출 연결입니다. 조회 목적에 필요하지 않은 삭제 권한은 제공하지 않고 필요한 범위를 확인합니다. “MCP가 연결되면 모든 작업이 자동 허용”은 오답입니다.

확인 질문과 해설

① 알림에 응답 id가 있나요? 오늘 initialized 알림은 응답을 기대하지 않습니다. ② null은 도구 오류인가요? 존재하는 기록의 미측정 값일 수 있습니다. ③ 실제로 연결됐는지는 무엇을 보고 확인하나요? 실제 요청과 대응 응답, 실행 범위를 남겨야 합니다.

호스트는 사용자가 요청하는 AI 앱 전체이고, 클라이언트는 그 안에서 특정 서버와 메시지를 주고받는 연결 부분입니다. 서버는 도구와 입력 규칙을 제공하고 실행 결과를 돌려줍니다. 오늘 예제의 클라이언트는 미리 쓴 요청만 보내며 실제 AI 호스

트나 모델이 판단하는 과정을 구현하지 않습니다.

뜻과 비유. MCP는 AI 앱이 외부 도구나 자료를 정해진 메시지 형식으로 사용하도록 연결하는 프로토콜입니다. 서버는 기능을 제공하고 클라이언트는 요청을 전달합니다.

왜 필요한가. React의 화면 구성과 MCP의 도구 통신을 같은 역할로 생각하지 않도록 경계를 구분합니다.

예를 따라 읽기. 제공 클라이언트는 `get_reading`이라는 읽기 도구를 요청하고 서버는 가상 센서 기록을 반환합니다.

햇갈리는 반례. 오늘의 `mcp_client.py`는 저장된 요청을 보내는 프로그램이며 AI 모델이나 실제 AI 호스트가 아닙니다.

직접 적용. 역할 카드의 네 요소 읽기 → 화면 역할과 도구 통신 역할 나누기 → 클라이언트·서버·데이터 파일 연결하기 → 실제 AI 호출이 없는 조건 쓰기

설명할 질문. MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요?

막혔을 때. MCP를 모든 AI 서비스의 동의어로 보지 않고 오늘 메시지를 주고 받는 두 프로그램을 정확히 지정합니다.

입력·답변·확인 순서는 [실습지의 A13](#)과 [활동 원문](#)에 있습니다.

MCP 필수 읽기와 참고 범위

필수: 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

강사 참고·선택: 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

A13 · 직접 해보기

A13 · MCP의 호스트·클라이언트·서버

목적

React의 화면 구성과 MCP의 도구 통신을 같은 역할로 생각하지 않도록 경계를 구분합니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답안과 작업 복사본을 사용합니다. 역할 카드와 `requests.jsonl`의 초기화·첫 도구 요청, `recorded_responses.json`의 성공·미측정·`isError`를 필수로 읽습니다. 전체 구현·모든 줄 해설은 강사 참고·선택입니다.

입력: [role_cards.md](#), [mcp_scope.md](#), [mcp_client.py](#), [mcp_server.py](#), [A13_input.md](#)

작업: [A13_notes.md](#)

실제 입력

제공 클라이언트는 `get_reading`이라는 읽기 도구를 요청하고 서버는 가상 센서 기록을 반환합니다.

단계

1. 역할 카드의 네 요소 읽기
2. 화면 역할과 도구 통신 역할 나누기
3. 클라이언트·서버·데이터 파일 연결하기
4. 실제 AI 호출이 없는 조건 쓰기

예상 결과

역할과 실제 실행 범위 표

확인 방법

MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요?

오류 해결

MCP를 모든 AI 서비스의 동의어로 보지 않고 오늘 메시지를 주고 받는 두 프로그램을 정확히 지정합니다.

힌트

MCP는 AI 앱이 외부 도구나 자료를 정해진 메시지 형식으로 사용하도록 연결하는 프로토콜입니다. 서버는 기능을 제공하고 클라이언트는 요청을 전달합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: MCP 서버가 웹 화면을 자동으로 만드는 것은 왜 아닌가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

MCP 필수 읽기와 참고 범위

필수: 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id:method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

강사 참고·선택: 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A15 · 실제 STUDIO 도구 호출

뜻과 비유. STUDIO는 한 프로그램이 다른 프로그램의 표준 입력과 표준 출력으로 메시지를 교환하는 방법입니다. 네트워크 계정 없이 로컬 자식 프로세스를 실행할 수 있습니다.

왜 필요한가. 저장된 예시 문장을 읽는 것과 실제 서버가 응답한 기록을 구분하면서 작은 프로토콜을 관찰합니다.

예를 따라 읽기. `python code/mcp_client.py`는 서버를 실행해 S01의 22.5, S02의 null, S99의 도구 오류를 받습니다.

헷갈리는 반례. S02의 null은 통신 실패가 아닙니다. S99의 `isError:true`와 값 없음은 서로 다릅니다.

직접 적용. 완성 클라이언트와 서버 줄 해설 읽기 → 클라이언트 명령 실행 또는 강사 결과 비교하기 → id3·4·5의 값과 오류 표시 찾기 → 응답 예와 실제 실행 여부를 분리 기록하기

설명할 질문. null 응답과 도구 오류 응답은 무엇이 다른가요?

막혔을 때. 서버를 단독 실행하면 입력을 기다릴 수 있습니다. 기본 활동은 제공 클라이언트 명령으로 실행합니다.

입력·답변·확인 순서는 [실습지의 A15](#)와 [활동 원문](#)에 있습니다.

MCP 필수 읽기와 참고 범위

필수: 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id:method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

강사 참고·선택: 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

A15 · 직접 해보기

A15 · 실제 STUDIO 도구 호출

목적

저장된 예시 문장을 읽는 것과 실제 서버가 응답한 기록을 구분하면서 작은 프로토콜을 관찰합니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 역할 카드와 requests.json의 초기화·첫 도구 요청, recorded_responses.json의 성공·미측정·isError를 필수로 읽습니다. 전체 구현·모든 줄 해설은 강사 참고·선택입니다.

입력: [mcp_client.py](#), [mcp_server.py](#), [requests.json](#), [readings.json](#), [A15_input.md](#)

작업: [A15_notes.md](#)

실제 입력

python code/mcp_client.py는 서버를 실행해 S01의 22.5,S02의 null,S99의 도구 오류를 받습니다.

단계

1. 완성 클라이언트와 서버 줄 해설 읽기
2. 클라이언트 명령 실행 또는 강사 결과 비교하기
3. id3·4·5의 값과 오류 표시 찾기
4. 응답 예와 실제 실행 여부를 분리 기록하기

예상 결과

성공·미측정·오류 호출 관찰 표

확인 방법

null 응답과 도구 오류 응답은 무엇이 다른가요?

오류 해결

서버를 단독 실행하면 입력을 기다릴 수 있습니다. 기본 활동은 제공 클라이언트 명령으로 실행합니다.

힌트

STDIO는 한 프로그램이 다른 프로그램의 표준 입력과 표준 출력으로 메시지를 교환하는 방법입니다. 네트워크 계정 없이 로컬 자식 프로세스를 실행할 수 있습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: null 응답과 도구 오류 응답은 무엇이 다른가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____

- 예상과 달랐던 점·복구·남은 질문: _____

MCP 필수 읽기와 참고 범위

필수: 역할 카드에서 호스트·클라이언트·서버·데이터를 구별합니다. `requests.jsonl` 첫 세 줄의 초기화·알림·도구 목록과 네 번째 줄의 `method/id/params`를 읽습니다. `recorded_responses.json`에서 S01의 숫자, S02의 미측정, S99의 `isError`를 대조합니다. 제공 클라이언트 실행 또는 강사 실행 관찰 후 수행 방법을 표시합니다.

필수 코드 확인은 `mcp_client.py` 67행(입력 메시지와 서버 실행), `mcp_server.py` 1617행(id·method), 2022행(초기화), 3443행(도구 요청과 결과)입니다. 구현 문법 전체를 풀이하지 않고 메시지의 역할과 입출력 대응을 확인합니다. 줄 번호는 이번 제공본 기준입니다.

강사 참고·선택: 서버 전체의 예외 처리, 표준 입출력, subprocess 구현과 모든 줄 해설. 전체 코드를 새로 작성하는 것은 필수가 아닙니다. 실제 AI 호스트 호출·인증·HTTP 연결은 이번 관찰 범위에 포함되지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: JSON-RPC 요청과 초기화

[이 교시의 다른 예와 코드 해설 열기](#)

6교시 · 역할 연결과 결과 확인

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A17 · 작은 변경의 동료 확인

뜻과 비유. 변경 검수는 입력한 값·바뀐 파일·예상 화면·실제 결과를 짝이 다시 확인하는 것입니다. 내용·모양·데이터·통신을 각기 다른 근거로 봅니다.

왜 필요한가. 화면의 모양이 좋아졌어도 도구에 올바른 입력을 보냈는지, 자료형을 유지했는지는 따로 확인해야 합니다.

예를 따라 읽기. HTML 제목,CSS 색,React 장소·온도,MCP sensor_id를 각각 한 줄의 변경 기록으로 남깁니다.

햇갈리는 반례. CSS 색 변경을 센서 상태로 기록하거나 모의 버튼을 실제 측정이라고 부르지 않습니다.

직접 적용. 작업 파일과 관찰 표 교환하기 → 네 역할 중 각 한 개 근거 확인하기 → 새로고침과 클릭 결과 구분하기 → 설명이 모호한 한 곳 보완하기

설명할 질문. 같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요?

막혔을 때. 짝의 코드를 바로 수정하지 말고 대상 파일과 기대한 결과를 먼저 설명합니다.

입력·답칸·확인 순서는 [실습지의 A17](#)와 [활동 원문](#)에 있습니다.

A17 · 직접 해보기

A17 · 작은 변경의 동료 확인

목적

화면의 모양이 좋아졌어도 도구에 올바른 입력을 보냈는지, 자료형을 유지했는지는 따로 확인해야 합니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [index.html](#), [style.css](#), [data.js](#), [requests.jsonl](#), [A17_input.md](#)

작업: [A17_notes.md](#), [index.html](#), [style.css](#), [data.js](#)

실제 입력

HTML 제목, CSS 색, React 장소·온도, MCP sensor_id를 각각 한 줄의 변경 기록으로 남깁니다.

단계

1. 작업 파일과 관찰 표 교환하기
2. 네 역할 중 각 한 개 근거 확인하기
3. 새로고침과 클릭 결과 구분하기
4. 설명이 모호한 한 곳 보완하기

예상 결과

네 요소별 입력·결과 검토 표

확인 방법

같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요?

오류 해결

짜의 코드를 바로 수정하지 말고 대상 파일과 기대한 결과를 먼저 설명합니다.

힌트

변경 검수는 입력한 값·바뀐 파일·예상 화면·실제 결과를 짝이 다시 확인하는 것입니다. 내용·모양·데이터·통신을 각기 다른 근거로 봅니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 같은 화면에서 서로 다른 역할의 변경을 어떻게 확인하나요? → ____
- 입력 파일과 내가 바꾼 위치: ____
- 실행 전 예상 결과: ____
- 실제 관찰 결과와 확인 방법: ____
- 예상과 달랐던 점·복구·남은 질문: ____

수정 전 예상: _____

확인한 결과와 이유: _____

A18 · 결과 설명과 선택 확장

뜻과 비유. 학습 결과는 수정한 작은 부분의 이유와 결과를 설명하는 것입니다. 전체 앱이나 MCP 서버를 독립 작성하는 능력을 오늘의 필수 목표로 삼지 않습니다.

왜 필요한가. 네 기술의 이름만 기억하는 것보다 어느 문제를 어떤 요소가 맡는지 말할 수 있어야 다음 대시보드로 연결됩니다.

예를 따라 읽기. 필수 결과는 제목·색·React 데이터 수정과 MCP 요청·응답 한 쌍의 설명입니다.

햇갈리는 반례. 패키지 설치를 못 했다고 개념 설명과 제공 번들 수정까지 실패로 판정하지 않습니다.

직접 적용. 수정한 파일 목록 정리하기 → 실제로 실행한 경로 쓰기 → 네 역할에 대한 새 질문 답하기 → 남은 환경·자동 연결 조건을 인계하기

설명할 질문. React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요?

막혔을 때. 설치나 외부 호스트 연결이 막히면 오프라인 번들과 저장된 응답으로 동일 개념을 확인하고 실행 범위를 명시합니다.

입력·답칸·확인 순서는 [실습지의 A18](#)와 [활동 원문](#)에 있습니다.

공식 근거와 원본

확인일 2026-09-26. 정의는 짧게 요약하고 활동·데이터·설명은 수업용으로 새로 작성했습니다.

- [HTML 기초](#): 태그·요소·속성의 의미
- [DOM 선택](#): 요소 선택 동작
- [React 상태](#): 상태와 setter 동작
- [React 화면 시작](#): DOM 루트에 컴포넌트 렌더링
- [MCP 수명주기](#): 명시 버전의 initialize와 initialized
- [MCP STUDIO](#): 한 줄 JSON-RPC·표준입출력
- [MCP 도구](#): tools/list와 tools/call·입력 스키마

개념 보충 공식 근거 (2026-10-05 확인)

- [React JSX: 화면 구조를 적는 문법](#)
- [MDN 상자 모델](#)
- [MCP 2025-11-25 구조와 역할](#)
- [MCP 도구와 사용자 통제](#)

A18 · 직접 해보기

A18 · 결과 설명과 선택 확장

목적

네 기술의 이름만 기억하는 것보다 어느 문제를 어떤 요소가 맡는지 말할 수 있어야 다음 대시보드로 연결됩니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [role_cards.md](#), [mcp_scope.md](#), [A18_input.md](#)

작업: [A18_notes.md](#)

실제 입력

필수 결과는 제목·색·React 데이터 수정과 MCP 요청·응답 한 쌍의 설명입니다.

단계

1. 수정한 파일 목록 정리하기
2. 실제로 실행한 경로 쓰기
3. 네 역할에 대한 새 질문 답하기
4. 남은 환경·자동 연결 조건을 인계하기

예상 결과

수정 묶음과 역할·검증·미확인 범위 보고

확인 방법

React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요?

오류 해결

설치나 외부 호스트 연결이 막히면 오프라인 번들과 저장된 응답으로 동일 개념을 확인하고 실행 범위를 명시합니다.

힌트

학습 결과는 수정한 작은 부분의 이유와 결과를 설명하는 것입니다. 전체 앱이나 MCP 서버를 독립 작성하는 능력을 오늘의 필수 목표로 삼지 않습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: React 상태·CSS 색·MCP 응답은 각각 무엇을 바꾸나요? → _____

- 입력 파일과 내가 바꾼 위치: _____

- 실행 전 예상 결과: _____

- 실제 관찰 결과와 확인 방법: _____

- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 네 요소의 연결과 말은 일

이 교시의 다른 예와 코드 해설 열기

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.
저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: work/A17_notes.md / work/A18_notes.md

세 가지 확인 질문

1. HTML, CSS, JavaScript가 화면에서 맡는 일을 각각 예로 설명해 보세요.

내 설명: _____

2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.

내 설명: _____

3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.

내 설명: _____